DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

AZURE DEVOPS AND CI/CD:

1. How can you implement CI/CD pipelines using Azure DevOps for deploying applications and services?

1. **Source Control:** Push your code to **Azure Repos** or **GitHub** to maintain version control.
2. **Continuous Integration (CI):**
    - Set up a **CI pipeline** that automatically **builds, tests, and validates** the code whenever changes are committed.
    - This ensures that the code is always in a deployable state.
3. **Continuous Deployment (CD):**
    - Create a **CD pipeline** to deploy your application or services to environments like **Azure Data Factory, Azure Functions, Databricks, or Synapse Analytics**.
    - You can configure stages for **Dev, QA, and Production** with approval gates.
4. **Pipeline Configuration:**
    - Pipelines can be defined using **YAML files** for versioning and automation or via the **classic UI editor** for a visual approach.
    - You can integrate tasks such as **unit tests, security scans, or database migrations** as part of the pipeline.

2. How can CI/CD practices improve the reliability and consistency of data pipelines in data engineering?

1. Error Reduction: Automated testing ensures that changes do not break existing pipelines, minimizing manual errors.
2. Early Issue Detection: Integration issues, data schema changes, or transformation errors are caught before reaching production.
3. Consistent Deployments: CI/CD ensures that the same version of code and configurations is deployed across Dev, QA, and Production, maintaining workflow consistency.
4. Faster Iterations: Data engineers can implement new transformations or features confidently, knowing deployments are safe and repeatable.

5. Scalability & Monitoring: Pipelines can include automated checks, logging, and alerts, enhancing overall operational reliability.

3. What are the main components and key features of Azure DevOps?

Azure DevOps provides a **comprehensive suite of tools** to manage the entire software development lifecycle. Its key components are:

1. **Azure Repos:** Provides **Git-based version control** to manage source code with branching, pull requests, and code reviews.
2. **Azure Pipelines:** Supports **CI/CD automation** to build, test, and deploy applications across multiple environments.
3. **Azure Boards:** Enables **Agile project tracking** using Kanban boards, Scrum boards, backlogs, and work items.
4. **Azure Artifacts:** Facilitates **package management**, allowing teams to create, host, and share packages such as **NuGet, npm, or Maven**.
5. **Azure Test Plans:** Provides **manual and automated testing** capabilities to ensure application quality before deployment.

4. How can you manage dependencies between multiple data pipelines in a CI/CD workflow?

Managing dependencies between data pipelines ensures that workflows execute in the correct sequence and that downstream pipelines receive valid data.

1. **Orchestration Tools:** Use tools like **Apache Airflow**, **Azure Data Factory**, or **Prefect** to orchestrate pipeline execution.
2. **DAGs (Directed Acyclic Graphs):** Define pipelines as DAGs to specify the order in which tasks should run, ensuring dependencies are respected.
3. **Triggers and Dependencies:**
    - In **Airflow**, you can set **task dependencies** using >> or set_upstream/set_downstream.
    - In **ADF**, use **pipeline dependencies** and **tumblings/trigger-based executions**.

4. **CI/CD Integration:** Ensure that the **pipeline code and configuration** are version-controlled and deployed together, so dependency rules remain consistent across **Dev, QA, and Production** environments.

5. What is the purpose of pipelines in Azure DevOps, and how are they used in software delivery?

Pipelines in Azure DevOps are used to **automate the software delivery process**, ensuring consistent and reliable builds, tests, and deployments. They provide the following capabilities:

1. **Build Automation:** Compile and package code automatically whenever changes are committed.
2. **Testing:** Run **unit tests, integration tests, and quality checks** to validate code before deployment.
3. **Deployment:** Automate the deployment of applications or services to **Dev, QA, or Production environments**.
4. **Pipeline Definitions:**
    - **YAML Pipelines:** Code-based, version-controlled, and reusable.
    - **Classic Pipelines:** Visual editor for configuring tasks and stages without code.
5. **Integration with DevOps Tools:** Pipelines can pull code from **Azure Repos, GitHub, or Bitbucket** and integrate with **Azure Boards, Artifacts, and Test Plans** for end-to-end delivery.
6. **Best Practices:** Include **approval gates, environment variables, and artifact versioning** to enhance reliability.

6. How do you monitor and track the performance of data pipelines after deployment in a CI/CD workflow?

1. **Monitoring Tools:** Use tools like **Prometheus, Grafana, Azure Monitor, or Log Analytics** to collect metrics such as execution time, data throughput, error rates, and resource utilization.
2. **Alerts and Notifications:** Configure alerts for **performance anomalies, failures, or delays** so the team can respond proactively.
3. **Logging:** Maintain detailed logs for pipeline runs to help diagnose issues and trace data flow.

4. **Dashboards:** Visualize pipeline metrics in dashboards to track trends, identify bottlenecks, and ensure SLAs are met.
5. **Integration with CI/CD:** Integrate monitoring into CI/CD pipelines to **automatically trigger alerts** or remediation scripts when certain thresholds are crossed.

7. How do you handle dependencies and versioning in Azure DevOps pipelines?

Managing dependencies and versioning ensures that builds are consistent and reproducible across environments. Key practices include:

1. **Package Management:** Use **Azure Artifacts** to host, manage, and share packages (e.g., NuGet, npm, Python packages).
2. **Versioning:** Apply **semantic versioning** (e.g., 1.0.0) to build artifacts to track changes and ensure backward compatibility.
3. **Dependency Locking:**
    - For Python projects, use **requirements.txt** to lock package versions.
    - For Node.js projects, use **package.json** with **package-lock.json** to ensure consistent dependencies.
4. **Pipeline Integration:** Configure pipelines to automatically restore dependencies from Azure Artifacts or external repositories before building or deploying applications.
5. **Best Practices:**
    - Maintain separate feeds for **stable** and **pre-release** versions.
    - Automate version bumps in CI/CD pipelines to avoid conflicts.

AZURE FUNCTION:

1. What types of triggers can Azure Functions use, and how do they work?

1. **HTTP Trigger:** Executes the function in response to **HTTP requests**. Used for APIs or webhooks.
2. **Timer Trigger:** Runs the function on a **schedule** using CRON expressions. Useful for scheduled tasks like nightly ETL jobs.

3. **Blob Trigger:** Executes when a **file is added or modified in Azure Blob Storage**. Ideal for file processing pipelines.
4. **Queue Trigger:** Executes when a **message is added to an Azure Storage Queue**. Useful for decoupled, asynchronous processing.
5. **Event Hub Trigger:** Responds to **events from Azure Event Hubs**, often used in streaming or telemetry scenarios.
6. **Service Bus Trigger:** Executes when a **message arrives in a Service Bus queue or topic**. Used for enterprise messaging workflows.
7. **Cosmos DB Trigger:** Executes when there are **changes in a Cosmos DB collection**, supporting real-time updates.

2. What are Azure Functions, and how do they differ from traditional web services or APIs?

Azure Functions are **serverless, event-driven compute services** that let you run code without managing infrastructure. Key characteristics include:

1. **Serverless:** You don't provision or manage servers; Azure handles scaling automatically.
2. **Event-driven:** Functions are triggered by events such as **HTTP requests, timers, blobs, queues, Event Hubs, or Cosmos DB changes**.
3. **Stateless and Short-lived:** Each function execution is independent, making it highly scalable.
4. **Cost-effective:** You only pay for the execution time and resources consumed, unlike always-on web services.
5. **Lightweight and Task-specific:** Ideal for small, discrete tasks rather than long-running processes.

Differences from Traditional Web Services:

- Traditional APIs require **managing servers, scaling, and uptime**; Functions are automatically scaled and managed.
- Functions are **event-driven and short-lived**, whereas traditional web services often handle continuous requests.
- Deployment of Functions is **simpler** for specific tasks, making them more agile for microservices or serverless architectures.

3. What are input and output bindings in Azure Functions, and how do they differ?

Bindings in Azure Functions simplify interaction with external resources without writing boilerplate code. They automatically **connect functions to data sources or services**.

1. **Input Binding:**
    - Provides data to the function from an external source.
    - Examples: Reading a message from **Azure Queue**, fetching a blob from **Azure Blob Storage**, or querying **Cosmos DB**.
    - The function automatically receives this data as a parameter.
2. **Output Binding:**
    - Sends data from the function to an external destination.
    - Examples: Writing processed data to **Blob Storage**, sending a message to a **Queue**, or updating a **Cosmos DB collection**.
    - The function writes to the binding without explicitly handling connection code.

Key Difference:

- **Input bindings** are **data sources for the function**.
- **Output bindings** are **targets where the function sends data**.

4. How do you deploy and manage Azure Functions in production environments?

1. **Deployment Options:**
    - **Azure DevOps Pipelines:** Automate build, test, and deployment of functions to different environments.
    - **GitHub Actions:** CI/CD workflows to deploy functions directly from a repository.
    - **Manual Deployment:** Deploy via **Visual Studio Code** or the **Azure Portal** for small projects or testing.
2. **Management and Monitoring:**
    - Use **Azure Portal** to monitor function executions, view logs, and manage triggers.

- Enable **Application Insights** to track performance metrics, exceptions, and request telemetry.
- Configure **scaling, retries, and environment settings** to ensure robust operation.

5. How does Azure Functions handle parallel execution of multiple requests or events?

Azure Functions supports **parallel and scalable execution** to handle multiple requests simultaneously:

1. **Stateless Design:** Each function execution is **independent and stateless**, allowing multiple instances to run concurrently without affecting each other.
2. **Dynamic Scaling:** Azure automatically **scales out** the number of function instances based on incoming events or request load.
3. **Event-driven Concurrency:**
   - For **HTTP triggers**, multiple requests are processed in parallel by different function instances.
   - For **queue or blob triggers**, messages or blobs are processed concurrently based on **batch size and max concurrency settings**.
4. **Configuration Options:**
   - You can configure **host.json** settings for **maxConcurrentRequests** or **batch size** to control concurrency.
5. **Example:**
   - An **Event Hub-triggered function** can process thousands of events in parallel, with Azure automatically provisioning additional function instances to handle the load.

6. What are some common use cases for Azure Functions in data engineering workflows?

1. **Triggering Jobs on Events:** Launching **Databricks or Synapse jobs** when files are uploaded to **Azure Blob Storage** or **Data Lake**.
2. **Data Cleaning and Validation:** Pre-processing incoming data to **ensure quality and consistency** before it is stored or processed further.

3. **Notifications and Alerts:** Sending automated **alerts or notifications** when certain events occur, such as pipeline failures or threshold breaches.
4. **Data Ingestion and Transformation:** Automatically ingesting, transforming, and routing data from source systems into **databases, data lakes, or analytics platforms**.

7. How do you implement CI/CD pipelines for deploying Azure Data Factory and Databricks solutions?

Implementing **CI/CD pipelines** for ADF and Databricks ensures **automated, consistent, and reliable deployments** across environments.

1. **Azure Data Factory (ADF):**
    - Use **Azure DevOps** to create **build and release pipelines** that automatically deploy **ADF pipelines, datasets, linked services, and triggers**.
    - The build pipeline exports ADF ARM templates, and the release pipeline deploys them to **Dev, QA, or Production** environments.
2. **Azure Databricks:**
    - Integrate **Databricks notebooks and jobs** with **Git version control** (e.g., GitHub or Azure Repos).
    - Use **Azure DevOps pipelines** to automate **testing, validation, and deployment** of notebooks, libraries, and other Databricks artifacts.
3. **Benefits:**
    - Ensures **consistency across environments** and reduces **manual errors**.
    - Supports **automation, testing, and versioning** for both ADF pipelines and Databricks workflows.

AZURE KEY VAULT:

1. **What is Azure Key Vault and why is it used?**

Azure Key Vault is a **cloud service for securely storing and managing secrets, keys,**

and certificates. It provides centralized **secret management** for applications, helping to protect sensitive information like API keys, passwords, and cryptographic keys.

Key points to mention in an interview:

- **Secrets:** Store and retrieve sensitive data securely.
- **Keys:** Manage encryption keys for data protection.
- **Certificates:** Manage SSL/TLS certificates.
- **Benefits:**
    - Reduces security risks by avoiding hard-coded secrets.
    - Integrates with **Azure AD** for fine-grained access control.
    - Provides **audit logs** for compliance.
    - Supports **hardware security modules (HSM)** for enhanced key protection.

2. How can you securely manage secrets, keys, and certificates using Azure Key Vault?

1. **Secure Storage:** Store **secrets** (passwords, API keys, connection strings), **certificates**, and **encryption keys** securely in the vault.
2. **Integration with Azure Services:** Azure Key Vault can be integrated with **Azure DevOps pipelines, Azure Functions, Databricks, and other Azure services** to avoid hardcoding secrets in code, notebooks, or configuration files.
3. **Access Control:**
    - Use **Azure Role-Based Access Control (RBAC)** and **Key Vault access policies** to restrict access to only authorized users or services.
    - Supports **Managed Identities** to allow services to securely retrieve secrets without storing credentials.
4. **Audit and Compliance:** Logs all access attempts and operations in **Azure Monitor**, providing traceability and regulatory compliance.

3. **What is the difference between secrets, keys, and certificates in Azure Key Vault?**

Azure Key Vault manages **three types of sensitive data**, each with a different purpose:

Type	Purpose	Example
Secrets	Store sensitive text-based information like passwords, connection strings.	Database password, API key
Keys	Store cryptographic keys used for encryption, decryption, or signing.	RSA, AES keys for data encryption
Certificates	Manage SSL/TLS certificates for secure communication.	HTTPS certificates for a web app

4. How can Azure Key Vault be integrated with other Azure services for secure secret management?

1. **Azure Data Factory (ADF):**
    - In Linked Services, select **"Use Azure Key Vault"** to fetch credentials such as database passwords or storage keys at runtime.
2. **Azure Functions:**
    - Use a **Managed Identity** to access secrets from Key Vault programmatically, ensuring credentials are never stored in code.
3. **Azure Databricks:**
    - Mount secrets using either a **Databricks-backed secret scope** or an **Azure Key Vault-backed secret scope** for secure access within notebooks and jobs.
4. **Other Azure Services:**
    - **Logic Apps, Synapse, and App Services** can also retrieve secrets securely via Key Vault integration.
5. **Best Practices:**
    - Use **Key Vault references** in configuration files or environment variables.
    - Monitor access via **Azure Monitor and diagnostic logs**.

5. How can Azure Monitor be used to track and analyze the performance of Azure services?

Azure Monitor provides comprehensive monitoring and observability for Azure resources, enabling you to track performance, detect issues, and troubleshoot effectively. Key steps include:

1. **Telemetry Collection:** Collect metrics and logs from services like **Azure Data Factory, Synapse, Azure Functions, App Services**, and more.
2. **Log Analytics Workspace:** Store logs in a **Log Analytics workspace** and use **Kusto Query Language (KQL)** to analyze trends, performance, and failures.
3. **Alerts and Notifications:** Configure **metric-based or log-based alerts** for events like **pipeline failures, high latency, or resource throttling**. Alerts can trigger **emails, webhooks, or automation runbooks**.
4. **Integration with Application Insights:** For deeper diagnostics, integrate with **Application Insights** to monitor request traces, dependencies, exceptions, and user metrics.

DATA MODELLING:

1. How would you design a simple star schema data warehouse for a grocery store?

A **data warehouse** for a grocery store can be designed using a **star schema**, which includes a central **fact table** and connected **dimension tables** for easy analysis.

Fact Table:

- **Fact_Sales** – captures transactional data:
  transaction_id, product_id, store_id, date_id, quantity_sold, revenue

Dimension Tables:

1. **Dim_Product** – product details:
   product_id, name, category, brand
2. **Dim_Store** – store details:
   store_id, location, store_type
3. **Dim_Date** – time attributes for analysis:
   date_id, date, week, month, year

Key Points:

- **Star Schema:** Fact table at the center, dimensions connected directly for simplicity and query performance.

- **Fact Table Granularity:** Each row represents a single sales transaction.
- **Use Cases:** Supports **sales analysis**, **revenue reporting**, and **trend insights** across products, stores, and time.

2. What are the different types of data warehouse schemas, and how do you decide which one to use for a data model?

1. **Star Schema:**
    - **Structure:** Central fact table surrounded by denormalized dimension tables.
    - **Advantages:** Simple, easy to understand, optimized for reporting and analytics.
    - **Use Case:** Best for straightforward reporting and dashboards where performance is critical.
2. **Snowflake Schema:**
    - **Structure:** Dimension tables are normalized into multiple related tables.
    - **Advantages:** Reduces data redundancy, handles hierarchies in dimensions effectively.
    - **Use Case:** Suitable when dimension tables are large or have complex hierarchies.
3. **Galaxy Schema (Fact Constellation):**
    - **Structure:** Multiple fact tables sharing common dimension tables.
    - **Advantages:** Supports multiple business processes in a single warehouse.
    - **Use Case:** Useful when analyzing different processes or events that share dimensions (e.g., sales and inventory).

3. What are fact tables and dimension tables in data modeling, and how are they used?

In **data modeling** for data warehouses, tables are classified into **fact tables** and **dimension tables** to organize data efficiently for analytics:

1. **Fact Table:**

- Stores **quantitative, measurable data** such as sales amount, revenue, or units sold.
- Contains **foreign keys** linking to dimension tables.
- Serves as the **central table** in star and snowflake schemas.

2. **Dimension Table:**
   - Stores **descriptive attributes** that provide context for facts, such as **product name, store location, date, or customer details**.
   - Used for **filtering, grouping, and slicing data** in reports and analytics.

4. What is the difference between a database, a data warehouse, and a data lake, and when would you use each?

These three types of storage serve **different purposes** in data management and analytics:

1. **Database:**
   - **Purpose:** Designed for **OLTP (Online Transaction Processing)**.
   - **Data Type:** Structured data, supporting real-time operations like **CRUD (Create, Read, Update, Delete)**.
   - **Use Case:** Applications such as **banking systems, e-commerce apps, or ERP** systems where transactional data is stored.
2. **Data Warehouse:**
   - **Purpose:** Optimized for **OLAP (Online Analytical Processing)**.
   - **Data Type:** Structured, cleaned, and transformed data.
   - **Use Case:** Reporting and analytics, dashboards (e.g., **Power BI, Tableau**) aggregating historical data for business insights.
3. **Data Lake:**
   - **Purpose:** Centralized storage for **large volumes of raw data**.
   - **Data Type:** Can store **structured, semi-structured, and unstructured data**.
   - **Use Case:** Big data processing, machine learning, and advanced analytics (e.g., ingesting logs, images, IoT data).

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

5. What is database normalization, and why is it important in data management?

Normalization is the process of **organizing data in a database** to reduce redundancy and ensure data integrity. Key points include:

1. **Process:**
    - Break down large tables into smaller, related tables.
    - Establish relationships between tables using **primary and foreign keys**.
    - Apply **normal forms** (1NF, 2NF, 3NF, etc.) to structure data efficiently.
2. **Purpose / Importance:**
    - **Eliminates Data Redundancy:** Prevents storing the same data in multiple places.
    - **Improves Data Integrity:** Ensures consistency and accuracy of data.
    - **Prevents Anomalies:** Avoids **update, insert, or delete anomalies** in OLTP systems.
    - **Efficient Storage:** Uses storage efficiently by minimizing duplicate data.

6. What is the difference between ER (Entity-Relationship) modeling and dimensional modeling in data systems?

1. **ER (Entity-Relationship) Modeling:**
    - Primarily used for **OLTP (transactional) systems**.
    - Focuses on **reducing redundancy** and **maintaining data integrity** through **normalization**.
    - Organizes data into **entities and relationships**.
2. **Dimensional Modeling:**
    - Primarily used for **OLAP (analytical) systems** like data warehouses.
    - Focuses on **query performance and reporting efficiency**.
    - Uses **fact tables** (quantitative data) and **dimension tables** (descriptive data).
    - Includes schema designs like **star schema, snowflake schema, or galaxy schema**

7. What is denormalization in database design, and when is it appropriate to use?

Denormalization is the process of **combining tables or adding redundant data** to reduce the number of joins and improve query performance. Key points include:

1. **Purpose:**
    - Optimizes **read-heavy operations** by reducing complex joins.
    - Improves **query performance** in reporting or analytics scenarios.
2. **When to Use:**
    - Primarily in **OLAP (analytical) systems** and **data warehouses**, where **fast query response** is more important than storage efficiency.
    - Useful when frequent reporting queries require joining multiple normalized tables.
3. **Trade-offs:**
    - Increased **data redundancy**, which can lead to **update anomalies** if not carefully managed.
    - Requires careful **ETL processes** to keep denormalized data consistent.

8. How do you handle Slowly Changing Dimensions (SCD) in data modeling, and what are the different types?

Slowly Changing Dimensions (SCD) are used to manage **changes in dimension data over time** in a data warehouse, ensuring accurate historical analysis.

1. **SCD Type 1 – Overwrite:**
    - Simply **overwrite the old value** with the new value.
    - **Pros:** Simple to implement.
    - **Cons: History is lost**, so past reports may not reflect original data.
2. **SCD Type 2 – Add New Row:**
    - Maintain history by **adding a new row** for the changed data, including **effective and expiry dates** or **version numbers**.
    - **Pros:** Preserves full history, ideal for analytics and trend reporting.
    - **Cons:** Slightly more complex to manage.

3. **SCD Type 3 – Add New Column:**
    - Add a **new column** to store the previous value alongside the current value.
    - **Pros:** Preserves some history.
    - **Cons:** Limited to **one or two historical changes**, rarely used in practice.

9. What are surrogate keys in dimensional modeling, and why are they used?

Surrogate keys are **unique, system-generated identifiers** (typically integers) used in **dimension tables** instead of relying on natural or business keys.

Key Points:

1. **Uniqueness:** Each row in the dimension table gets a **unique surrogate key**, independent of business logic.
2. **Benefits:**
    - **Simplifies handling Slowly Changing Dimensions (SCDs):** New versions of a dimension row can have a new surrogate key, while historical data remains intact.
    - **Avoids issues with changing natural keys:** Even if a business key changes, the surrogate key remains stable.
    - **Improves performance:** Joins between fact and dimension tables are faster using integer surrogate keys.

10. How do you optimize data models for performance and scalability in data warehouses?

Optimizing data models ensures **efficient query performance, scalability, and maintainability** in analytical systems. Key strategies include:

1. **Schema Design:**
    - Prefer **star schema** over snowflake schema when **query performance** is critical, as it reduces joins.
    - Avoid unnecessary joins that can slow down queries.
2. **Indexing and Partitioning:**

- Use **appropriate indexing** on fact and dimension tables to speed up query access.
- Partition large tables by **date or other high-cardinality columns** to improve read performance.

3. **Data Storage Formats:**
   - Use **columnar storage formats** like **Parquet or ORC** for analytics workloads to reduce I/O and storage costs.
4. **Data Management:**
   - **Archive or purge old data** periodically to maintain performance and manage storage efficiently.

11. What is a galaxy schema in data warehousing, and when is it used?

A **Galaxy Schema**, also called a **Fact Constellation Schema**, is a type of **dimensional modeling** used in data warehouses.

Key Points:

1. **Structure:**
   - Contains **multiple fact tables** that share **dimension tables**.
   - Each fact table represents a **different business process** (e.g., sales, inventory, shipments).
   - Dimension tables provide **common descriptive attributes** for the fact tables.
2. **Advantages:**
   - Supports **complex analytics** across multiple business processes.
   - Reuses dimension tables, reducing redundancy.
   - Scales well for **enterprise-level warehouses**.
3. **Use Case:**
   - Ideal when an organization needs to analyze **different events or processes** that share dimensions, like **Sales Facts** and **Inventory Facts** both using **Product, Store, and Date** dimensions.

12. What is the role of metadata in data modeling and data architecture?

Metadata is essentially **"data about data"** and plays a critical role in **data modeling, architecture, and governance**.

Key Points:

1. **Describes Structure:**
    - Provides details about **tables, columns, data types, constraints**, and relationships in the data model.
2. **Tracks Lineage:**
    - Shows **where data originates**, how it is transformed, and where it is used across systems.
3. **Provides Business Context:**
    - Documents **definitions, metrics, and business rules** to help users understand the data.
4. **Enables Governance and Auditing:**
    - Supports **data governance**, **impact analysis**, **compliance**, and **audit trails**.
5. **Improves Discovery and Reuse:**
    - Facilitates **data discovery** and allows teams to **reuse existing datasets** effectively.

13. What is a data mart, and how does it differ from a data warehouse?

A **Data Mart** is a **focused subset of a data warehouse**, designed to serve the needs of a **specific business unit or department**, such as **Sales, Marketing, or Finance**.

Key Points:

1. **Scope:**
    - **Data Mart:** Limited to a **specific domain or department**.
    - **Data Warehouse:** Enterprise-wide, integrating data from multiple sources across the organization.
2. **Purpose:**
    - **Data Mart:** Optimized for **department-specific reporting and analytics**.

    - **Data Warehouse:** Supports **organization-wide analytics, decision-making, and historical analysis**.

3. **Complexity and Build Time:**
    - **Data Mart:** Simpler and **faster to build**.
    - **Data Warehouse:** Larger and more **complex to implement and maintain**.
4. **Integration:**
    - **Data Mart:** Can be **dependent** on the main data warehouse or built independently (standalone).
    - **Data Warehouse:** Acts as a **central repository** for all organizational data.

14. What is a junk dimension, and when would you use it?

A **Junk Dimension** is a small dimension table that **combines miscellaneous flags or attributes** (like Yes/No flags, statuses) that do not belong in other dimensions.

- Reduces clutter in the fact table.
- Simplifies schema design by consolidating unrelated attributes.
  Example: Order_Flag, Priority, Shipment_Status combined into a single Junk Dimension for the Orders fact table.

15. **What is a factless fact table, and why is it used?**

A **Factless Fact Table** captures **events or occurrences** without numeric measures.

- Useful for tracking **attendance, student enrollments, or product promotions**.
- Example: Fact_Student_Attendance with foreign keys Student_ID, Class_ID, Date_ID records attendance events.